Task 1.1: Fully Visible Sigmoid Belief Network (FVSBN)

CHEN Jiaqi jchen3785 CS5494 Assignment 1

1.1 Model · 1.2 Experimental setup · 1.3 Results · 1.4 Discussion · 1.5 Limitations

1.1 Model

The assignment asks for image generation with logistic regression, which is the fully visible sigmoid belief network. The idea is to write the joint distribution of an image as a product of one conditional per pixel,

$$p(x) \;=\; \prod_{i=1}^{784} p\big(x_i \mid x_{<i}\big), \tag{1.1}$$

where $x_{<i}$ are the pixels that come before i in a fixed generation order. For a binarised 28×28 image the vector has 784 entries and the natural order is row-major, $i = 28\,\mathrm{row} + \mathrm{col}$. Every conditional is a logistic regression on all previous pixels:

$$p\big(x_i = 1 \mid x_{<i}\big) \;=\; \sigma\Big(w_i^{\top} x_{<i} + b_i\Big), \qquad \sigma(z) = \frac{1}{1+e^{-z}}. \tag{1.2}$$

Stacking the 784 rows w_i into a matrix $W \in \mathbb{R}^{784\times 784}$ gives the vector form used in `fvsbn.py`:

$$\mathrm{logits}(x) \;=\; (W \odot M)^{\top} x + b, \qquad M_{ij} = \begin{cases} 1, & j < i, \\ 0, & \text{otherwise}, \end{cases} \tag{1.3}$$

where $\odot$ is the element-wise product, so row i of W only sees pixels with index below i. The model has $784 \times 784 + 784 = 615{,}440$ parameters and no hidden layer. It is called *fully visible* because the only latent quantity is the pixel being predicted: there is no latent variable that sums over the whole image, so nothing has to be approximated. Training is ordinary supervised learning on the observed pixels with the negative log likelihood

$$\mathcal{L}_{\mathrm{NLL}} \;=\; -\sum_{i=1}^{784} \Big[\, x_i \log p_i + (1 - x_i) \log(1 - p_i) \,\Big], \tag{1.4}$$

which is exactly 784 binary cross entropies added together, averaged over the batch. Because every conditional only depends on earlier pixels, the likelihood can be evaluated in one masked matrix product, and samples can be drawn ancestrally: draw x_1, then x_2 from $p(x_2 \mid x_1)$, and so on for 28 sequential steps.

1.2 Experimental setup

MNIST was binarised deterministically with a threshold of 0.5, so a pixel is 1 when it is brighter than 0.5. The fixed threshold keeps the training set identical between runs, which matters for reproducing the numbers. The resulting images contain 13.2% ink on average in the training split and 13.4% in the test split. The 60,000 training images are used for training; the 10,000 test images are used only for the reported numbers. The test NLL is also the validation signal during training, which is fine here because there is nothing to tune on it (the learning rate and batch size were fixed once, see below).

The optimiser is Adam with learning rate 10^{-3}, batch size 128 and 50 epochs, seed 0. There is no weight decay, dropout or learning rate schedule. An RTX 3070 Laptop GPU was used, and the same code runs on CPU. Two metrics are reported: the negative log likelihood per image (sum over the 784 pixels) and bits per dimension,

$$\mathrm{bpd} \;=\; \frac{\mathcal{L}_{\mathrm{NLL}}}{784 \ln 2}, \tag{1.5}$$

so lower is better for both and they are defined on the same binarised data.

Three ablations check that the numbers are not an accident of one setting: the number of training images (1k, 5k, 20k, 60k, 30 epochs), the binarisation threshold (0.3, 0.5, 0.7, 20k images, 30 epochs) and the pixel order (row-major versus column-major, 20k images, 30 epochs).

1.3 Results

The main run converges quickly and then flattens out (Figure 1.1). The first epoch already reaches 0.386 bpd, which is close to the unconditional baseline, and by epoch 10 the model is at 0.164. The final test numbers are 75.75 nats per image, 0.1394 bpd and 96.0% pixel accuracy under teacher forcing. Training took 30 to 50 s depending on GPU load (about 1 s per epoch, 60,000 images), and drawing 100 images takes about 0.06 s in the median of five runs, up to 0.15 s when the GPU is busy, so it is well under a millisecond per image despite the 28 sequential steps.

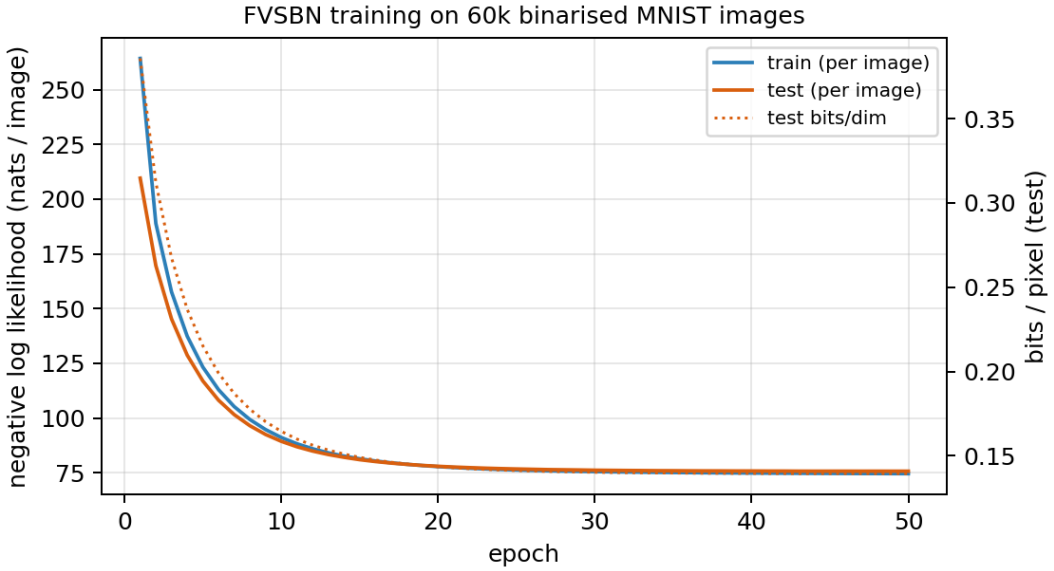


Figure 1.1: Training and test NLL per image; the right axis is the test bits per dimension.

The reference point is a model that ignores the prefix and predicts every pixel from its training marginal. That baseline costs 205.85 nats per image (0.3788 bpd), so the FVSBN removes about 63% of the baseline NLL. The gap between train and test NLL is small (74.6 versus 75.7 at the end), which says the linear model is underfitting rather than overfitting: more capacity, not more regularisation, is what this model needs.

Figure 1.2 shows four samples from the same model at epochs 1, 5, 10, 25 and 50. The first row is noise, by epoch 5 there are blobs, and from epoch 10 on the strokes of digits appear. Figure 1.3 shows 40 samples of the final model. The digits are recognisable but thick and sometimes smeared: the model reproduces the amount of ink (mean pixel 0.131 in the samples against 0.134 in the data) but not the fine structure, for example the thin diagonal of a 7 or the sharp corner of a 2.

Figure 1.4 looks at the same test digits in two ways. The middle column is the teacher-forced probability $p(x_i = 1 \mid x_{<i})$, which is mostly close to 0 or 1: given the true prefix the model is confident about the next pixel. The right column is the argmax of that probability, and it reconstructs the digit almost perfectly. So the per-pixel conditionals are good even though the joint samples are only rough. This is the usual exposure bias of autoregressive models: when sampling, the model conditions on its own earlier mistakes instead of the true pixels, and the small errors accumulate along the 784 steps.

The ablations are collected in Table 1.1 and Figure 1.5. The training set size is the dominant

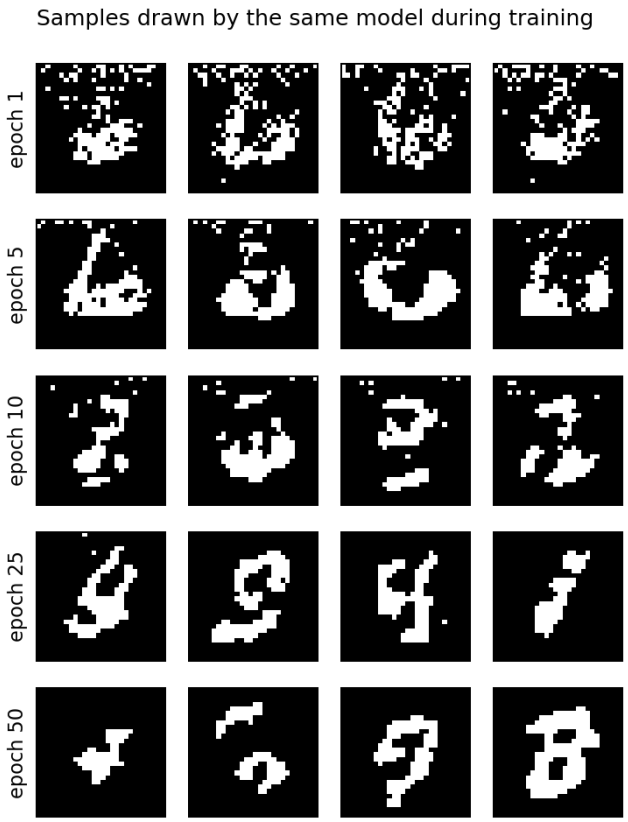


Figure 1.2: Samples drawn by the same model at epochs 1, 5, 10, 25 and 50.

factor: 1,000 images give 0.454 bpd, 5,000 give 0.289, 20,000 give 0.166 and 60,000 give 0.140. Most of the improvement happens before 20,000 images, and doubling to 60,000 only buys 0.015 bpd. The column-major order is slightly worse than row-major (0.1677 against 0.1656), which fits the fact that neighbouring pixels in a row are more correlated than pixels in a column for this kind of writing. The threshold rows need care: lowering the threshold to 0.3 adds ink and lowers bpd to 0.1604, raising it to 0.7 removes ink and gives 0.1717, but the data distribution is different in each case, so these three numbers only compare the model with itself at a fixed amount of ink.

Table 1.1: All runs, 30 epochs except the main run (50 epochs). Bits per dimension is $\mathcal{L}_{\mathrm{NLL}}/(784 \ln 2)$.

run	images	threshold	order	test NLL	test bpd
main	60000	0.5	row	75.75	0.1394
size 1k	1000	0.5	row	246.53	0.4537
size 5k	5000	0.5	row	156.94	0.2888
size 20k	20000	0.5	row	90.00	0.1656
size 60k	60000	0.5	row	76.19	0.1402
threshold 0.3	20000	0.3	row	87.19	0.1604
threshold 0.7	20000	0.7	row	93.32	0.1717
column-major	20000	0.5	col	91.14	0.1677

1.4 Discussion

The model does exactly what a logistic-regression generator can do. Every conditional is a linear function of the prefix, so the model can learn that a dark pixel is likely after a run of dark pixels, or



Figure 1.3: 40 samples of the final model.

that a curve continues in some direction, but it has no notion of a stroke being the same object at a different position. There is no weight sharing and no hidden unit that could detect a local pattern; each of the 784 rows of W has to learn on its own. That is also why it needs 615,440 parameters, about 4.5 times the number of values in a single image, and why increasing the training set still helps at 60,000 images.

The sampling experiments are the most interesting part of the section for the comparison with task 1.2. FVSBN reaches a decent bpd in about a minute of training, and the teacher-forced reconstructions are nearly perfect, but the joint samples are blurry. A hidden layer changes exactly this: it can represent non-linear interactions between pixels and reuse them at every position, which is the motivation for the neural autoregressive model in the next section. The course also covers estimators for models with latent units, but FVSBN does not need one: every variable is observed, so the likelihood is exact and the gradient is the gradient of a binary cross entropy.

Two smaller observations. First, the order of the pixels matters only a little in terms of NLL (0.1656 versus 0.1677), but it changes what a sample looks like: with column-major order the strokes are built from top to bottom, so errors in the first column are copied down the whole image. Second, the per-pixel conditionals are so confident that a naive argmax of the teacher-forced probabilities is a good reconstruction, which is why the model is sometimes described as a *deterministic* pixel predictor. The gap between that reconstruction and the samples is the price of sampling from the joint.

1.5 Limitations

The linear conditionals are the main limitation: for a given prefix the model predicts one Bernoulli probability per pixel, and a linear logit cannot describe curved or multi-modal conditional distributions. A second limitation is speed: sampling needs 28 sequential matrix products, so it is slow compared with a GAN generator that produces an image in one forward pass. A third is that the model is unconditional; generating a specific digit would need the label in the prefix. Adding a hidden layer (task 1.2) and conditioning on the class are the two obvious extensions, and the ablation table already shows where the gains would come from: more expressive conditionals, not more data.

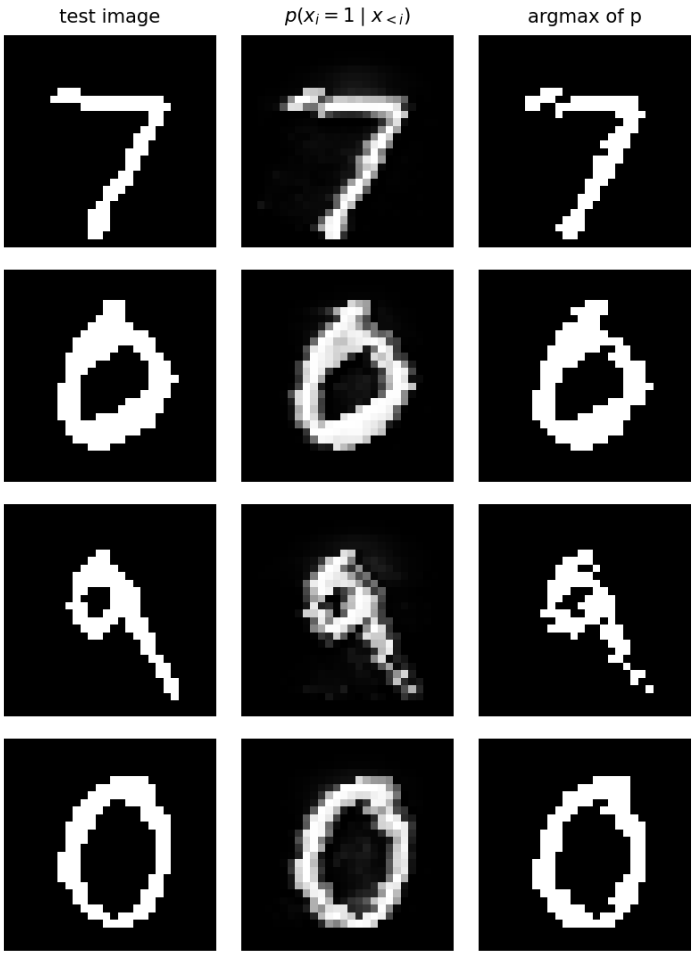


Figure 1.4: Test image, teacher-forced conditional probability and its argmax.

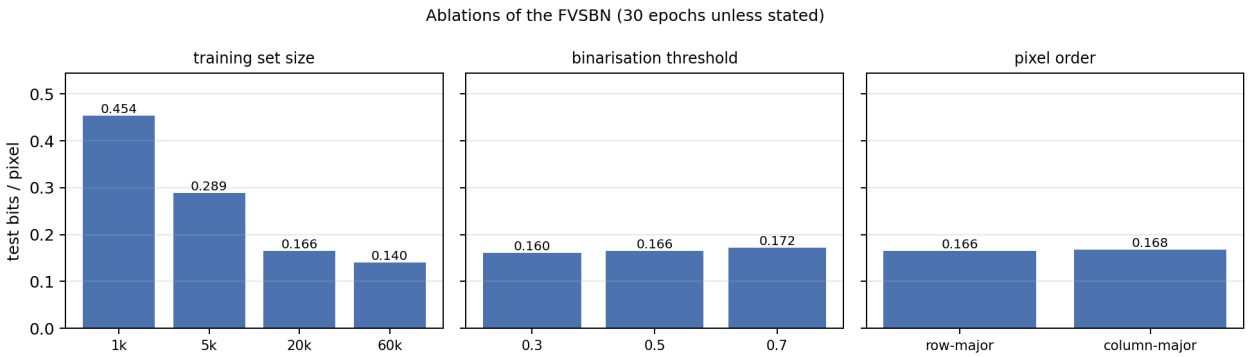


Figure 1.5: Ablations on training set size, binarisation threshold and pixel order.

CS5494 Assignment 1 - Task 1.2

Neural Autoregressive Density Estimator (NADE) on binarized MNIST

Name: Zhu Pengyu **EID:** pengyuzhu3

This notebook implements the neural autoregressive model required by Task 1.2. It follows Lecture 3 and uses the shared `data.py` supplied with Task 1.1 so that NADE and FVSBN use exactly the same MNIST split, binarization rule and row-major pixel order.

Run order:

1. Keep `QUICK_RUN=True` and run all model/training cells once as a smoke test.
2. Set `QUICK_RUN=False`, restart the kernel, and run all cells for the final run.
3. Set `RUN_ABLATION=True` only when the main run is working.

The notebook writes all generated artifacts under `build/`.

1. Imports and reproducibility

Only packages already used by Task 1.1 are required: PyTorch, torchvision, NumPy and Matplotlib.

In [1]:
```
import csv
import json
import math
import os
import random
import time

import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset

import data as D

SEED = 0
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Python/PyTorch environment ready")
```

```
print("PyTorch:", torch.__version__)
print("Device:", device)
if device.type == "cuda":
    print("GPU:", torch.cuda.get_device_name(0))

BUILD = "build"
RUNS = os.path.join(BUILD, "runs")
FIGS = os.path.join(BUILD, "figs")
os.makedirs(RUNS, exist_ok=True)
os.makedirs(FIGS, exist_ok=True)
```

```
Python/PyTorch environment ready
PyTorch: 2.14.0+cu130
Device: cuda
GPU: NVIDIA GeForce RTX 3060 Laptop GPU
```

2. Experiment configuration

`QUICK_RUN=True` is only for checking that the implementation works. It is **not** the final assignment experiment. For the final report, set it to `False`, restart the kernel and run the notebook again.

In [2]:
```
QUICK_RUN = False
RUN_ABLATION = False

if QUICK_RUN:
    N_TRAIN = 5000
    N_TEST = 2000
    EPOCHS = 3
    STAGE_EPOCHS = (1, 2, 3)
else:
    N_TRAIN = 60000
    N_TEST = 10000
    EPOCHS = 50
    STAGE_EPOCHS = (1, 5, 10, 25, 50)

DIM = 784
HIDDEN = 128
BATCH_SIZE = 64
LEARNING_RATE = 1e-3
THRESHOLD = 0.5
PIXEL_ORDER = "row"

print({
    "quick_run": QUICK_RUN,
    "n_train": N_TRAIN,
    "n_test": N_TEST,
    "epochs": EPOCHS,
    "hidden": HIDDEN,
    "batch_size": BATCH_SIZE,
    "learning_rate": LEARNING_RATE,
    "threshold": THRESHOLD,
    "pixel_order": PIXEL_ORDER,
    "seed": SEED,
})
```

```
{'quick_run': False, 'n_train': 60000, 'n_test': 10000, 'epochs': 50, 'hidden': 1
28, 'batch_size': 64, 'learning_rate': 0.001, 'threshold': 0.5, 'pixel_order': 'r
ow', 'seed': 0}
```

3. Shared MNIST preprocessing

Task 1.1 and Task 1.2 must use the same data contract:

- deterministic binarization with threshold 0.5;
- values exactly in `{0, 1}`;
- flattened row-major to 784 pixels;
- fixed subset seed 0 when a subset is used.

In [3]:
```
x_train_all, y_train_all = D.load_binarized(
    "train", threshold=THRESHOLD, mode="threshold", seed=SEED
)
x_test_all, y_test_all = D.load_binarized(
    "test", threshold=THRESHOLD, mode="threshold", seed=SEED
)

x_train, y_train = D.take_subset(
    x_train_all, y_train_all, N_TRAIN, seed=SEED
)
x_test, y_test = D.take_subset(
    x_test_all, y_test_all, N_TEST, seed=SEED
)

perm = D.pixel_order(PIXEL_ORDER)
x_train = D.apply_order(x_train, perm)
x_test = D.apply_order(x_test, perm)

assert x_train.dtype == np.float32
assert x_train.shape[1] == DIM
assert set(np.unique(x_train)).issubset({0.0, 1.0})

print("train:", x_train.shape, "test:", x_test.shape)
print("train ink fraction:", float(x_train.mean()))
print("test ink fraction:", float(x_test.mean()))

preview = D.undo_order(x_train[:16], perm)
D.show_grid(preview, rows=2, cols=8, title="Shared binarized MNIST training samp
plt.show()
```

```
  0%|          | 0.00/9.91M [00:00<?, ?B/s]

  0%|          | 32.8k/9.91M [00:00<01:07, 147kB/s]

  1%|          | 98.3k/9.91M [00:00<00:42, 229kB/s]

  2%|▏         | 164k/9.91M [00:00<00:38, 254kB/s]

  3%|▎         | 295k/9.91M [00:00<00:25, 379kB/s]

  5%|▍         | 459k/9.91M [00:01<00:18, 500kB/s]
```

```
  9%|▉          | 852k/9.91M [00:01<00:09, 921kB/s]
 17%|█▋         | 1.64M/9.91M [00:01<00:04, 1.75MB/s]
 26%|██▌        | 2.56M/9.91M [00:01<00:02, 3.01MB/s]
 33%|███▎       | 3.28M/9.91M [00:01<00:01, 3.64MB/s]
 42%|████▏      | 4.19M/9.91M [00:01<00:01, 4.78MB/s]
 52%|█████▏     | 5.11M/9.91M [00:02<00:00, 5.77MB/s]
 59%|█████▉     | 5.83M/9.91M [00:02<00:01, 3.66MB/s]
 84%|████████▍  | 8.32M/9.91M [00:02<00:00, 6.00MB/s]
100%|██████████| 9.91M/9.91M [00:02<00:00, 3.72MB/s]

  0%|          | 0.00/28.9k [00:00<?, ?B/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 103kB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 103kB/s]

  0%|          | 0.00/1.65M [00:00<?, ?B/s]
  2%|▏         | 32.8k/1.65M [00:00<00:12, 129kB/s]
  6%|▌         | 98.3k/1.65M [00:00<00:07, 221kB/s]
 12%|█▏        | 197k/1.65M [00:01<00:07, 197kB/s]
 20%|█▉        | 328k/1.65M [00:01<00:04, 302kB/s]
 34%|███▍      | 557k/1.65M [00:01<00:02, 491kB/s]
 60%|█████▉    | 983k/1.65M [00:01<00:00, 876kB/s]
 83%|████████▎ | 1.38M/1.65M [00:01<00:00, 1.13MB/s]
 91%|█████████ | 1.51M/1.65M [00:02<00:00, 775kB/s]
100%|██████████| 1.65M/1.65M [00:02<00:00, 691kB/s]

  0%|          | 0.00/4.54k [00:00<?, ?B/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 4.54MB/s]
train: (60000, 784) test: (10000, 784)
train ink fraction: 0.1322583109140396
test ink fraction: 0.13422946631908417
```

```
C:\Users\zpy\AppData\Local\Temp\ipykernel_4120\493140892.py:29: UserWarning: Figu
reCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
```

4. NADE model

For a binary image $x = (x_1, \ldots, x_D)$, NADE models

$$p(x) = \prod_{i=1}^{D} p(x_i \mid x_{<i}).$$

A shared hidden layer represents the prefix:

$$a_i = c + \sum_{j<i} W_{:,j} x_j, \qquad h_i = \sigma(a_i),$$

and the current Bernoulli probability is

$$p(x_i = 1 \mid x_{<i}) = \sigma(V_i^T h_i + b_i).$$

The shifted cumulative sum in `logits` guarantees that output i never sees the current or future pixels. Training is parallel over all target pixels; sampling remains sequential because each newly sampled pixel becomes part of the next prefix.

In [4]:
```python
class NADE(nn.Module):
    def __init__(self, dim=784, hidden=128):
        super().__init__()
        self.dim = dim
        self.hidden = hidden

        # Shared input-to-hidden weights and hidden-to-output weights.
        self.W = nn.Parameter(torch.empty(hidden, dim))
        self.V = nn.Parameter(torch.empty(dim, hidden))
        self.c = nn.Parameter(torch.zeros(hidden))
        self.b = nn.Parameter(torch.zeros(dim))

        nn.init.xavier_uniform_(self.W)
        nn.init.xavier_uniform_(self.V)

    def logits(self, x):
        # contribution[b, h, i] = W[h, i] * x[b, i]
        contribution = x.unsqueeze(1) * self.W.unsqueeze(0)
        prefix = torch.cumsum(contribution, dim=2)

        zero_prefix = torch.zeros(
            x.size(0), self.hidden, 1,
            device=x.device, dtype=x.dtype
        )

        # Shift by one position: prediction i only receives x_<i.
        hidden_input = self.c.view(1, -1, 1) + torch.cat(
            [zero_prefix, prefix[:, :, :-1]], dim=2
        )
        hidden_state = torch.sigmoid(hidden_input)
```

```
        return torch.einsum("bhd,dh->bd", hidden_state, self.V) + self.b

    def nll(self, x):
        return F.binary_cross_entropy_with_logits(
            self.logits(x), x, reduction="none"
        ).sum(dim=1)

    @torch.no_grad()
    def sample(self, n, device=None):
        device = device or self.b.device
        samples = torch.zeros(n, self.dim, device=device)
        hidden_input = self.c.unsqueeze(0).expand(n, -1).clone()

        for i in range(self.dim):
            hidden_state = torch.sigmoid(hidden_input)
            logit = (
                hidden_state * self.V[i].unsqueeze(0)
            ).sum(dim=1) + self.b[i]
            probability = torch.sigmoid(logit)
            current_pixel = torch.bernoulli(probability)
            samples[:, i] = current_pixel
            hidden_input = hidden_input + (
                current_pixel.unsqueeze(1) * self.W[:, i].unsqueeze(0)
            )

        return samples


model_check = NADE(dim=8, hidden=4)
toy_x = torch.bernoulli(torch.full((5, 8), 0.3))
assert model_check.logits(toy_x).shape == toy_x.shape
assert model_check.nll(toy_x).shape == (5,)
assert model_check.sample(3).shape == (3, 8)
print("NADE shape and sampling checks passed")
```

```
NADE shape and sampling checks passed
```

5. Evaluation, timing and training helpers

NLL is summed over all 784 pixels and averaged over images. Bits per dimension is

$$\text{bpd} = \frac{\text{NLL}}{784 \log 2}.$$

Sampling time follows the Task 1.1 protocol: one warm-up call, five timed repetitions, CUDA synchronization and the median time for 100 samples.

In [5]:
```
def parameter_count(model):
    return int(sum(p.numel() for p in model.parameters() if p.requires_grad))


def evaluate_nll(model, x, batch_size=128):
    model.eval()
    loader = DataLoader(
        TensorDataset(torch.from_numpy(np.ascontiguousarray(x)).float()),
        batch_size=batch_size,
        shuffle=False,
```

```
    )
    total = 0.0
    count = 0
    with torch.no_grad():
        for (batch_x,) in loader:
            batch_x = batch_x.to(device)
            total += model.nll(batch_x).sum().item()
            count += batch_x.size(0)
    return total / count


def time_sampling(model, n=100, repeats=5):
    model.eval()
    model.sample(n, device=device)
    if device.type == "cuda":
        torch.cuda.synchronize()

    measurements = []
    for _ in range(repeats):
        if device.type == "cuda":
            torch.cuda.synchronize()
        start = time.perf_counter()
        model.sample(n, device=device)
        if device.type == "cuda":
            torch.cuda.synchronize()
        measurements.append(time.perf_counter() - start)
    return float(np.median(measurements))


def train_nade(
    x_train,
    x_eval,
    hidden=128,
    epochs=50,
    batch_size=64,
    learning_rate=1e-3,
    stage_epochs=(1, 5, 10, 25, 50),
    stage_samples=4,
    seed=0,
    verbose=True,
):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

    model = NADE(dim=x_train.shape[1], hidden=hidden).to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

    train_tensor = torch.from_numpy(np.ascontiguousarray(x_train)).float()
    generator = torch.Generator().manual_seed(seed)
    loader = DataLoader(
        TensorDataset(train_tensor),
        batch_size=batch_size,
        shuffle=True,
        generator=generator,
    )

    history = {
```

```
        "epoch": [],
        "train_nll": [],
        "eval_nll": [],
        "eval_bpd": [],
    }
    stages = {}

    if device.type == "cuda":
        torch.cuda.synchronize()
    training_start = time.perf_counter()

    for epoch in range(1, epochs + 1):
        model.train()
        total = 0.0
        count = 0

        for (batch_x,) in loader:
            batch_x = batch_x.to(device)
            loss = model.nll(batch_x).mean()

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            total += loss.item() * batch_x.size(0)
            count += batch_x.size(0)

        train_nll = total / count
        eval_nll = evaluate_nll(model, x_eval, batch_size=batch_size)
        eval_bpd = eval_nll / (x_train.shape[1] * math.log(2.0))

        history["epoch"].append(epoch)
        history["train_nll"].append(train_nll)
        history["eval_nll"].append(eval_nll)
        history["eval_bpd"].append(eval_bpd)

        if epoch in stage_epochs:
            stages[epoch] = model.sample(
                stage_samples, device=device
            ).cpu().numpy()

        if verbose and (epoch == 1 or epoch % 5 == 0 or epoch == epochs):
            print(
                f"epoch {epoch:3d} | train NLL {train_nll:8.2f} | "
                f"eval NLL {eval_nll:8.2f} | eval bpd {eval_bpd:.4f}"
            )

    if device.type == "cuda":
        torch.cuda.synchronize()
    training_time = time.perf_counter() - training_start

    sample_time_100 = time_sampling(model, n=100, repeats=5)
    final_samples = model.sample(40, device=device).cpu().numpy()

    return {
        "model": model,
        "history": history,
        "stages": stages,
        "final_samples": final_samples,
        "training_time": float(training_time),
```

```
        "sample_time_100": sample_time_100,
        "parameters": parameter_count(model),
    }
```

6. Main experiment

The quick run verifies the complete pipeline. The final numbers used in the report must come from `QUICK_RUN=False`.

In [6]:
```
result = train_nade(
    x_train=x_train,
    x_eval=x_test,
    hidden=HIDDEN,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    learning_rate=LEARNING_RATE,
    stage_epochs=STAGE_EPOCHS,
    stage_samples=4,
    seed=SEED,
)

model = result["model"]
history = result["history"]

final_nll = history["eval_nll"][-1]
final_bpd = history["eval_bpd"][-1]

print("\nFinal experiment summary")
print("quick run:", QUICK_RUN)
print("test/eval NLL per image:", final_nll)
print("test/eval bits per dimension:", final_bpd)
print("parameters:", result["parameters"])
print("training time (s):", result["training_time"])
print("time for 100 samples (s):", result["sample_time_100"])
```

```
epoch   1 | train NLL   170.81 | eval NLL   128.83 | eval bpd 0.2371
epoch   5 | train NLL    86.75 | eval NLL    84.24 | eval bpd 0.1550
epoch  10 | train NLL    77.30 | eval NLL    76.77 | eval bpd 0.1413
epoch  15 | train NLL    74.11 | eval NLL    74.20 | eval bpd 0.1365
epoch  20 | train NLL    72.42 | eval NLL    72.90 | eval bpd 0.1342
epoch  25 | train NLL    71.34 | eval NLL    72.09 | eval bpd 0.1327
epoch  30 | train NLL    70.58 | eval NLL    71.49 | eval bpd 0.1316
epoch  35 | train NLL    70.00 | eval NLL    71.10 | eval bpd 0.1308
epoch  40 | train NLL    69.55 | eval NLL    70.77 | eval bpd 0.1302
epoch  45 | train NLL    69.17 | eval NLL    70.52 | eval bpd 0.1298
epoch  50 | train NLL    68.86 | eval NLL    70.28 | eval bpd 0.1293
Final experiment summary
quick run: False
test/eval NLL per image: 70.28168876953124
test/eval bits per dimension: 0.12933041307793913
parameters: 201616
training time (s): 244.13101360000292
time for 100 samples (s): 0.10700630000064848
```

7. Save the run and generate required figures

In [7]:
```
run_name = "nade_quick" if QUICK_RUN else "nade_main_60k_ep50"

run_payload = {
    "name": run_name,
    "model": "NADE",
    "quick_run": QUICK_RUN,
    "n_train": int(len(x_train)),
    "n_test": int(len(x_test)),
    "epochs": EPOCHS,
    "batch_size": BATCH_SIZE,
    "learning_rate": LEARNING_RATE,
    "hidden": HIDDEN,
    "threshold": THRESHOLD,
    "pixel_order": PIXEL_ORDER,
    "seed": SEED,
    "device": str(device),
    "gpu": torch.cuda.get_device_name(0) if device.type == "cuda" else "cpu",
    "parameters": result["parameters"],
    "training_time_s": result["training_time"],
    "sample_time_100_s": result["sample_time_100"],
    "final_nll": final_nll,
    "final_bpd": final_bpd,
    "history": history,
}

with open(os.path.join(RUNS, run_name + ".json"), "w", encoding="utf-8") as file
    json.dump(run_payload, file, indent=2)

np.savez_compressed(
    os.path.join(RUNS, run_name + ".npz"),
    final=result["final_samples"],
    perm=perm,
    **{f"stage_{epoch}": samples for epoch, samples in result["stages"].items()}
)

torch.save(model.state_dict(), os.path.join(RUNS, run_name + ".pt"))
print("Saved run:", run_name)
```

```
Saved run: nade_main_60k_ep50
```

In [8]:
```
fig, ax1 = plt.subplots(figsize=(7, 4))
ax1.plot(history["epoch"], history["train_nll"], label="train NLL")
ax1.plot(history["epoch"], history["eval_nll"], label="eval NLL")
ax1.set_xlabel("epoch")
ax1.set_ylabel("NLL per image")
ax1.grid(alpha=0.25)
ax1.legend(loc="upper right")

ax2 = ax1.twinx()
ax2.plot(history["epoch"], history["eval_bpd"], "k--", label="eval bpd")
ax2.set_ylabel("bits per dimension")

plt.title("NADE training and evaluation loss")
fig.tight_layout()
loss_path = os.path.join(FIGS, "fig_nade_loss.png")
fig.savefig(loss_path, dpi=180, bbox_inches="tight")
plt.show()
print(loss_path)
```

```
build\figs\fig_nade_loss.png
```

```
C:\Users\zpy\AppData\Local\Temp\ipykernel_4120\4282775652.py:17: UserWarning: Fig
ureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
```

In [9]:
```
stage_epochs_available = sorted(result["stages"])
stage_images = np.concatenate(
    [D.undo_order(result["stages"][epoch], perm) for epoch in stage_epochs_avail
    axis=0,
)
stage_titles = [f"epoch {epoch}" for epoch in stage_epochs_available]
stage_path = os.path.join(FIGS, "fig_nade_stages.png")
D.show_grid(
    stage_images,
    rows=len(stage_epochs_available),
    cols=4,
    path=stage_path,
    titles=stage_titles,
    title="NADE samples during training",
)
plt.figure(figsize=(10, 2.2 * len(stage_epochs_available)))
plt.imshow(plt.imread(stage_path))
plt.axis("off")
plt.show()

final_images = D.undo_order(result["final_samples"], perm)
final_path = os.path.join(FIGS, "fig_nade_samples.png")
D.show_grid(
    final_images,
    rows=5,
    cols=8,
    path=final_path,
    title="Final NADE samples",
)
plt.figure(figsize=(10, 7))
plt.imshow(plt.imread(final_path))
plt.axis("off")
plt.show()

print(stage_path)
print(final_path)
```

```
C:\Users\zpy\AppData\Local\Temp\ipykernel_4120\1778453934.py:19: UserWarning: Fig
ureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
build\figs\fig_nade_stages.png
build\figs\fig_nade_samples.png
C:\Users\zpy\AppData\Local\Temp\ipykernel_4120\1778453934.py:33: UserWarning: Fig
ureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
```

8. Teacher-forced conditional predictions

This figure separates conditional prediction quality from free-running ancestral sampling. During teacher forcing, each prediction receives the true prefix; during sampling, it receives the model's earlier samples, so errors may accumulate.

In [10]:
```
model.eval()
example_tensor = torch.from_numpy(x_test[:6]).float().to(device)
with torch.no_grad():
    probabilities = torch.sigmoid(model.logits(example_tensor)).cpu().numpy()

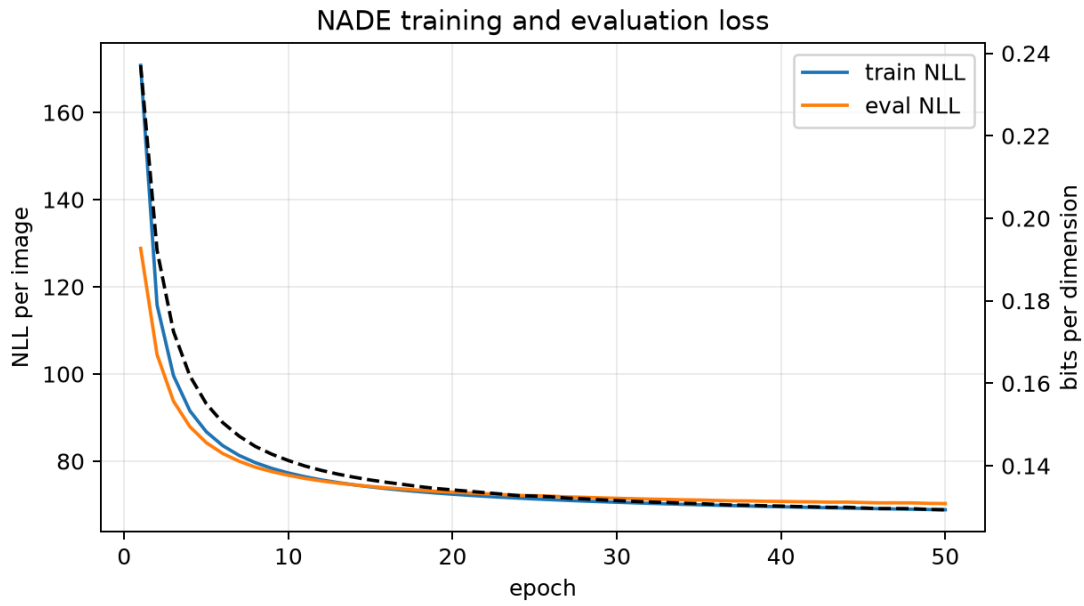
original = D.undo_order(x_test[:6], perm).reshape(-1, 28, 28)
probability_maps = D.undo_order(probabilities, perm).reshape(-1, 28, 28)
argmax_maps = (probability_maps >= 0.5).astype(np.float32)

fig, axes = plt.subplots(6, 3, figsize=(7, 12))
for row in range(6):
    axes[row, 0].imshow(original[row], cmap="gray", vmin=0, vmax=1)
    axes[row, 1].imshow(probability_maps[row], cmap="gray", vmin=0, vmax=1)
    axes[row, 2].imshow(argmax_maps[row], cmap="gray", vmin=0, vmax=1)
    for col in range(3):
        axes[row, col].axis("off")
axes[0, 0].set_title("test image")
axes[0, 1].set_title("conditional probability")
axes[0, 2].set_title("thresholded prediction")
fig.tight_layout()
pred_path = os.path.join(FIGS, "fig_nade_predmap.png")
fig.savefig(pred_path, dpi=180, bbox_inches="tight")
plt.show()
print(pred_path)
```

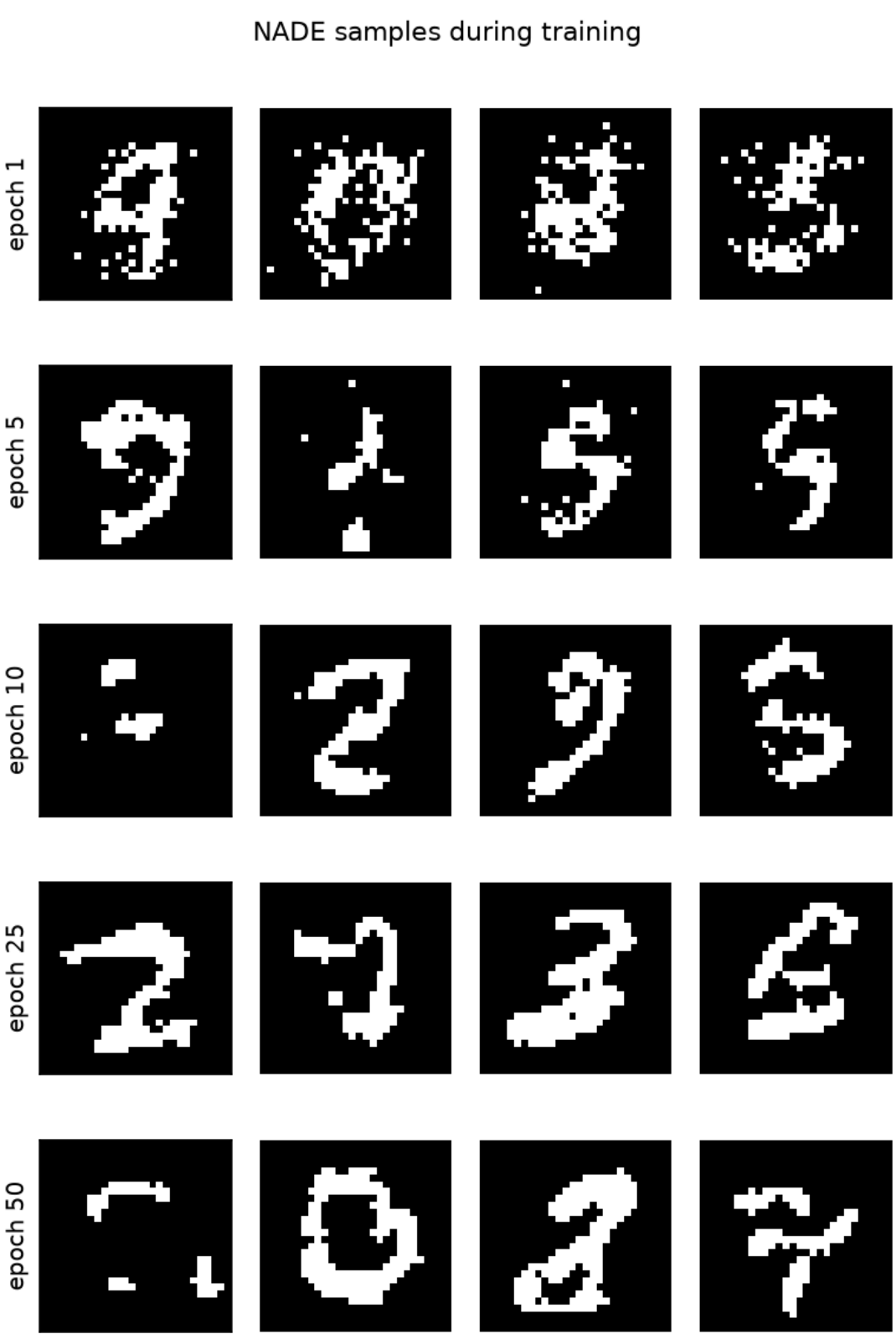
```
build\figs\fig_nade_predmap.png
```

```
C:\Users\zpy\AppData\Local\Temp\ipykernel_4120\1791546170.py:23: UserWarning: Fig
ureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
```

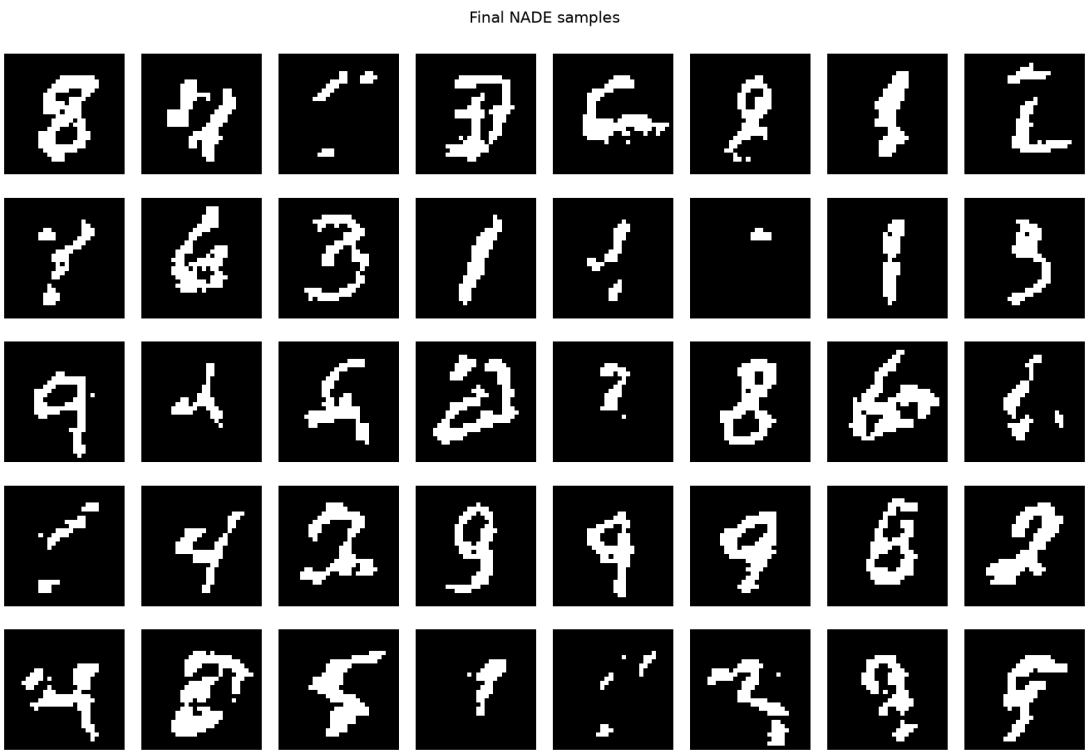
Loss curve



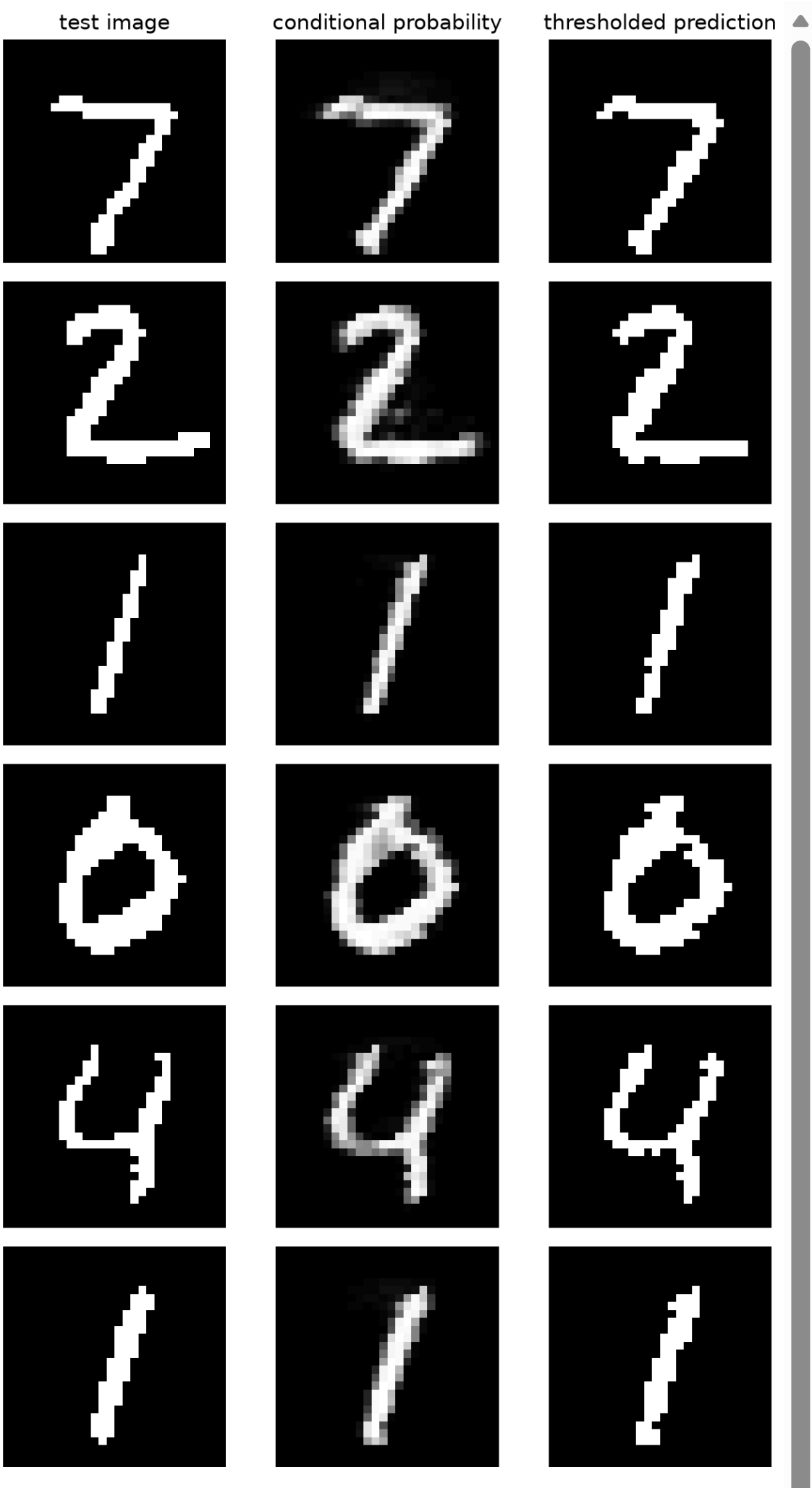
Samples during training



Final samples



Teacher-forced conditional predictions


test image
conditional probability
thresholded prediction

9. Hidden-width ablation

This is intentionally disabled for the first run. When the main model works, set `RUN_ABLATION=True`. The final ablation uses 20,000 training images, 30 epochs and hidden widths 64, 128 and 256 while keeping all other settings fixed.

In [11]:
```
ablation_results = []

if RUN_ABLATION:
    if QUICK_RUN:
        ablation_n_train = 2000
        ablation_n_test = 1000
        ablation_epochs = 2
    else:
        ablation_n_train = 20000
        ablation_n_test = 10000
        ablation_epochs = 30

    x_ablation, y_ablation = D.take_subset(
        x_train_all, y_train_all, ablation_n_train, seed=SEED
    )
    x_ablation_test, _ = D.take_subset(
        x_test_all, y_test_all, ablation_n_test, seed=SEED
    )
    x_ablation = D.apply_order(x_ablation, perm)
    x_ablation_test = D.apply_order(x_ablation_test, perm)

    for hidden_size in (64, 128, 256):
        print("\nHidden size:", hidden_size)
        current = train_nade(
            x_train=x_ablation,
            x_eval=x_ablation_test,
            hidden=hidden_size,
            epochs=ablation_epochs,
            batch_size=BATCH_SIZE,
            learning_rate=LEARNING_RATE,
            stage_epochs=(),
            stage_samples=0,
            seed=SEED,
            verbose=False,
        )

        ablation_results.append({
            "hidden": hidden_size,
            "parameters": current["parameters"],
            "test_nll": current["history"]["eval_nll"][-1],
            "test_bpd": current["history"]["eval_bpd"][-1],
            "training_time_s": current["training_time"],
            "sample_time_100_s": current["sample_time_100"],
        })
        print(ablation_results[-1])

    csv_path = os.path.join(BUILD, "nade_hidden_ablation.csv")
    with open(csv_path, "w", newline="", encoding="utf-8") as file:
        writer = csv.DictWriter(file, fieldnames=ablation_results[0].keys())
        writer.writeheader()
```

```
        writer.writerows(ablation_results)
    print("Saved:", csv_path)
else:
    print("Ablation skipped. Set RUN_ABLATION=True when ready.")
```

```
Ablation skipped. Set RUN_ABLATION=True when ready.
```

In [12]:
```
if ablation_results:
    hidden_sizes = [row["hidden"] for row in ablation_results]
    bpd_values = [row["test_bpd"] for row in ablation_results]

    plt.figure(figsize=(6, 4))
    plt.bar([str(value) for value in hidden_sizes], bpd_values)
    plt.xlabel("hidden units")
    plt.ylabel("test bits per dimension")
    plt.title("NADE hidden-width ablation")
    plt.grid(axis="y", alpha=0.25)
    ablation_path = os.path.join(FIGS, "fig_nade_hidden_ablation.png")
    plt.tight_layout()
    plt.savefig(ablation_path, dpi=180, bbox_inches="tight")
    plt.show()
    print(ablation_path)
```

10. Task 1 comparison and handoff files

Only FVSBN and NADE have exact likelihoods on the same binarized data, so their bpd values can be compared directly. GAN losses must not be placed in the same numerical comparison.

In [13]:
```
fvsbn_reference = {
    "test_bpd": 0.1394,
    "test_nll": 75.75,
    "parameters": 615440,
    "training_time_s": "30-50",
    "sample_time_100_s": 0.06,
}

print("FVSBN reference:", fvsbn_reference)
print("NADE current run:", {
    "test_bpd": final_bpd,
    "test_nll": final_nll,
    "parameters": result["parameters"],
    "training_time_s": result["training_time"],
    "sample_time_100_s": result["sample_time_100"],
})

comparison_row = (
    f"| NADE (section 4) | {final_bpd:.4f} | {result['parameters']:,} | "
    f"{result['training_time']:.1f} s | {result['sample_time_100']:.3f} s | "
    "one hidden layer, shared nonlinear conditionals |"
)

report_values = (
    "# Task 1.2 handoff values\n\n"
    f"- Quick run: {QUICK_RUN}\n"
    "- Model: one-hidden-layer NADE\n"
    f"- Hidden units: {HIDDEN}\n"
    f"- Train/test images: {len(x_train)} / {len(x_test)}\n"
```

```
    f"- Epochs: {EPOCHS}\n"
    f"- Test NLL per image: {final_nll:.4f} nats\n"
    f"- Test bits per dimension: {final_bpd:.4f}\n"
    f"- Parameters: {result['parameters']:,}\n"
    f"- Training time: {result['training_time']:.3f} s\n"
    f"- Time for 100 samples: {result['sample_time_100']:.6f} s\n"
    "- Entry point: CS5494_Assignment1_Task2_NADE.ipynb\n\n"
    "## Comparison-table row\n\n"
    f"{comparison_row}\n"
)

handoff_path = os.path.join(BUILD, "task2_handoff_values.md")
with open(handoff_path, "w", encoding="utf-8") as file:
    file.write(report_values)

print(comparison_row)
print("Saved:", handoff_path)
```

```
FVSBN reference: {'test_bpd': 0.1394, 'test_nll': 75.75, 'parameters': 615440, 't
raining_time_s': '30-50', 'sample_time_100_s': 0.06}
NADE current run: {'test_bpd': 0.12933041307793913, 'test_nll': 70.2816887695312
4, 'parameters': 201616, 'training_time_s': 244.13101360000292, 'sample_time_100_
s': 0.10700630000064848}
| NADE (section 4) | 0.1293 | 201,616 | 244.1 s | 0.107 s | one hidden layer, sha
red nonlinear conditionals |
Saved: build\task2_handoff_values.md
```

11. Report Section 4: Neural Autoregressive Model

This section summarizes Task 1.2 using the completed 60,000-image, 50-epoch run above. Quick-run values are not used.

4.1 Model

The joint distribution of a binarized image is factorized in row-major pixel order as

$$p(\mathbf{x}) = \prod_{i=1}^{784} p(x_i \mid x_{<i}).$$

A one-hidden-layer Neural Autoregressive Density Estimator (NADE) is used to model these conditionals. For pixel i, its hidden representation and Bernoulli probability are

$$\mathbf{h}_i = \sigma\left(\mathbf{c} + \sum_{j<i} \mathbf{W}_{:,j} x_j\right), \qquad p(x_i = 1 \mid x_{<i}) = \sigma(\mathbf{V}_{i,:}\mathbf{h}_i + b_i).$$

The implementation computes all teacher-forced conditionals efficiently with a shifted cumulative sum. The shift is essential: prediction i receives only pixels $x_{<i}$ and never the current or future pixels. The model is trained by minimizing the exact Bernoulli negative log-likelihood (NLL), summed over 784 pixels and averaged over images. It contains **201,616 trainable parameters**. Generation is

exact ancestral sampling: pixels are sampled one at a time and the hidden pre-activation is updated after every sampled pixel, requiring 784 sequential decisions per image.

4.2 Experimental setup

MNIST is loaded through the shared `data.py` used by Task 1.1. All 60,000 training images and 10,000 test images are deterministically binarized using `pixel > 0.5`, flattened to 784 dimensions in row-major order, and stored as `float32`. This keeps the likelihood comparison with FVSBN consistent.

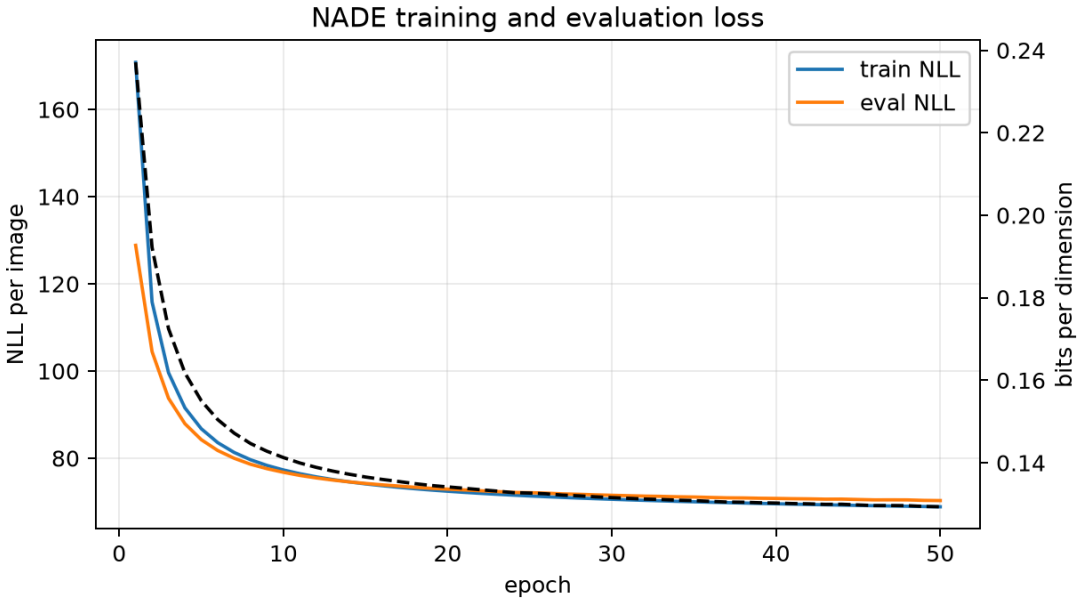
Setting	Value
Hidden units	128
Epochs	50
Batch size	64
Optimizer	Adam
Learning rate	0.001
Random seed	0
Pixel order	Row-major
Hardware	NVIDIA GeForce RTX 3060 Laptop GPU
Software	PyTorch 2.14.0+cu130

Bits per dimension is calculated as $\mathrm{bpd} = \mathrm{NLL}/(784 \log 2)$. Sampling time is measured for 100 images after one warm-up, with CUDA synchronization and the median of five repetitions.

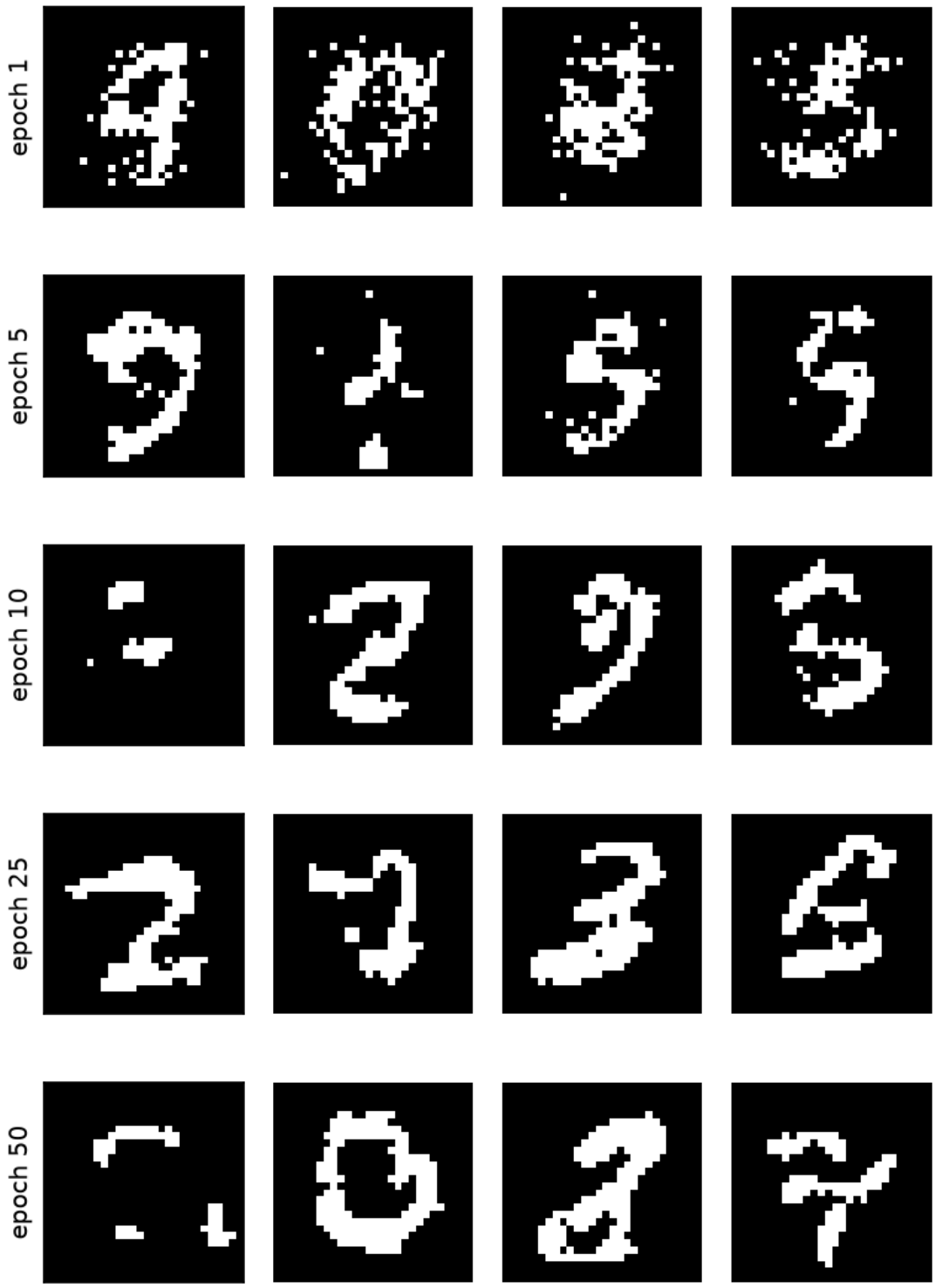
4.3 Results

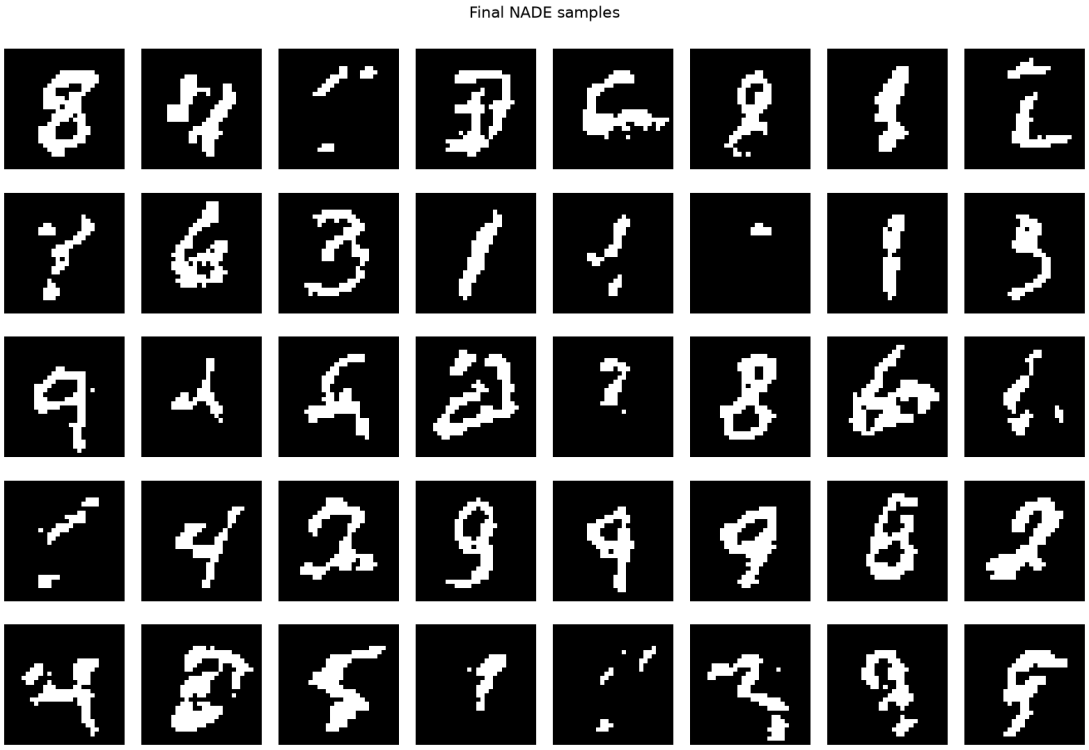
Metric	Formal-run result
Final train NLL	68.8562 nats/image
Test NLL	**70.2817 nats/image**
Test bits per dimension	**0.1293 bpd**
Train-test NLL gap	1.4255 nats/image
Trainable parameters	**201,616**
Training time	**244.13 s**
Time for 100 samples	**0.1070 s**

The model improved rapidly during the first ten epochs and then continued to improve more gradually. Test NLL decreased from 128.83 at epoch 1 to 84.24 at epoch 5, 76.77 at epoch 10, 72.09 at epoch 25, and 70.28 at epoch 50. The corresponding final test bpd was 0.1293. The train-test gap was only 1.43 nats, approximately 2% of the test NLL, so this run did not show severe overfitting.

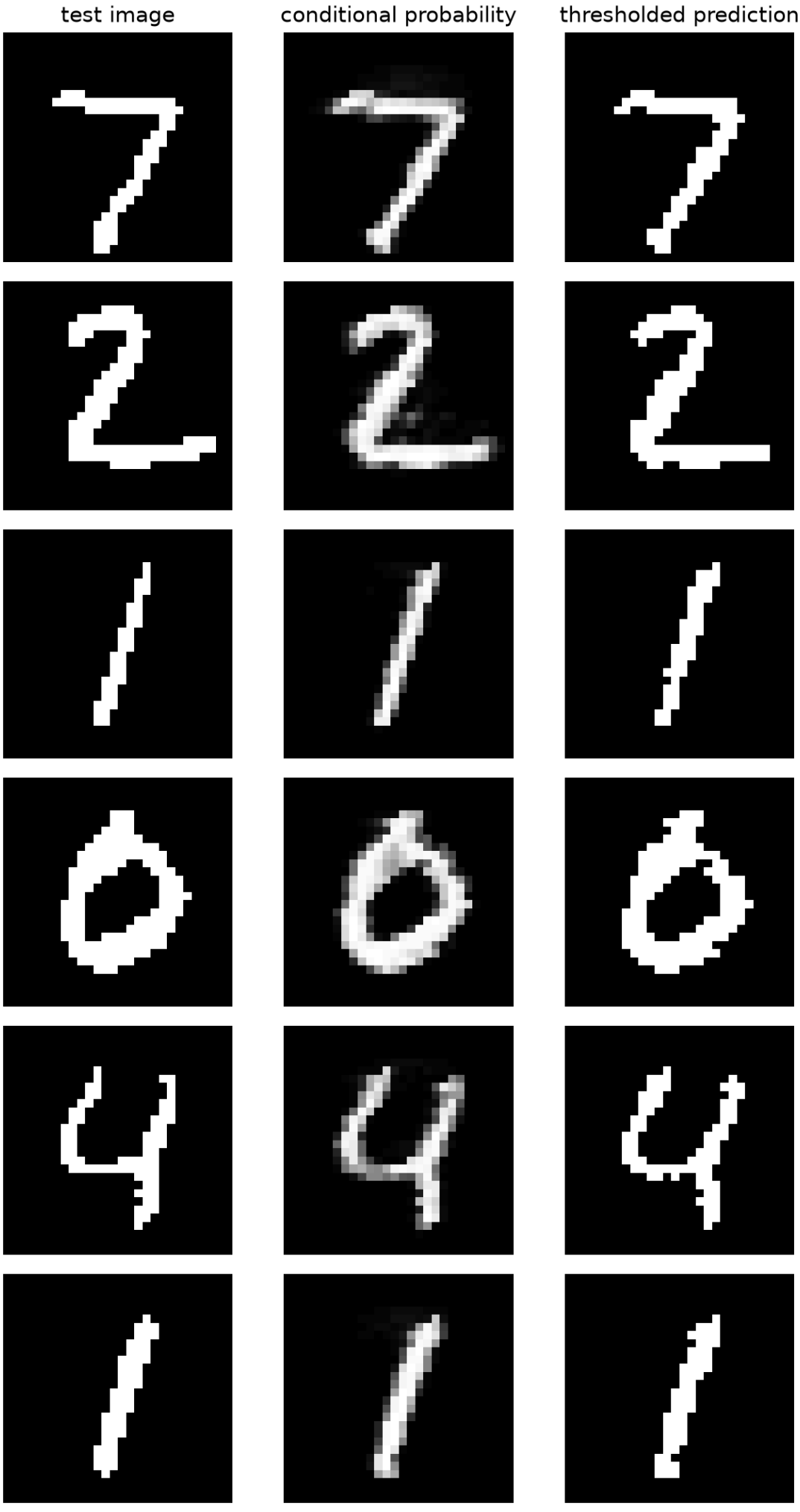


The staged samples show a clear qualitative progression. Epoch-1 samples contain substantial noise and disconnected pixels; recognizable digit structures begin to appear by epochs 5-10; and the epoch-25 and epoch-50 samples have more coherent strokes. The final sample grid contains several digit classes and writing styles, indicating useful diversity rather than obvious collapse. Most samples are recognizable, although some contain broken or overly thick strokes and a few are ambiguous between classes.


NADE samples during training
epoch 1
epoch 5
epoch 10
epoch 25
epoch 50



Teacher-forced probability maps reconstruct the test digits clearly, and their thresholded versions are close to the input images. They are cleaner than some free-running samples because each conditional receives the true prefix during teacher forcing, whereas an early sampling error becomes part of the context for all later pixels.


test image
conditional probability
thresholded prediction

4.4 Discussion

On the same binarized MNIST representation, NADE improved test NLL from the FVSBN result of 75.75 to 70.28 nats/image and improved bpd from 0.1394 to 0.1293, a reduction of approximately **7.2%**. NADE also used 201,616 parameters instead of 615,440, a reduction of approximately **67.2%**. These results support the value of the shared nonlinear hidden representation: it captures reusable dependencies among previous pixels more efficiently than independent linear conditional models.

The stronger likelihood does not make NADE cheaper in every respect. It required 244.13 s to train and 0.107 s to generate 100 images, whereas the available FVSBN records report 30-50 s and approximately 0.06 s. However, the FVSBN timing was collected on an RTX 3070 Laptop GPU and this NADE run used an RTX 3060 Laptop GPU, so the timing gap cannot be attributed solely to model architecture. Both autoregressive models also require **784 sequential pixel-generation steps**, not 28 steps.

4.5 Limitations

NADE generation is inherently sequential, making sampling slower than a parallel generator. Results can depend on the fixed row-major order, which does not explicitly preserve two-dimensional locality. Training uses true prefixes while generation uses previously sampled pixels, so errors can accumulate over the 784 steps. The model is unconditional and has no label control, and its fully connected structure does not exploit convolutional translation priors. Finally, the hidden-width ablation code is included but was not enabled in the formal run; therefore, no claim is made that 128 hidden units are optimal. A strict runtime comparison also requires rerunning all models on the same hardware and software stack.

Task 1.2 comparison-table row

Model	Test bpd	Parameters	Training time	100 samples	Notes
NADE	**0.1293**	**201,616**	**244.1 s**	**0.107 s**	One hidden layer; shared nonlinear conditionals

CS5494 Assignment 1 - Task1.3

- Name: Li Shuo
- EID: sli986

MNIST Handwritten Digit Generation with a Convolutional GAN

In this experiment, under the same budget of 20 training iterations, we compared the baseline model with a model using a lower learning rate and conducted experiments to isolate the effects of BatchNorm on the generator.

The baseline achieved **89.5% human recognizability** on 200 fixed evaluation samples, compared with **84.0%** for the lower learning rate and **82.0%** without generator BatchNorm. All three runs covered ten recognizable digit classes, but their class distributions were uneven. These findings describe one training seed and one human annotation pass.

In [ ]:
```
from report_utils import load_report_data, show_table, show_figure, show_checkpo
report = load_report_data()
```

1. Objective and experimental questions

The generator learns to transform Gaussian noise into a digit image, while the discriminator learns to distinguish training images from generated images. The experiment targets:

1. Whether the baseline produces recognizable and varied digits.
2. How reducing both learning rates affects results within a fixed budget.
3. How removing generator BatchNorm changes learning and final samples.

E0 is a preliminary debugging run. We use E1-E3 to compare.

2. Data preparation and loading

The input is **original grayscale MNIST**, with image shape $1 \times 28 \times 28$. 60,000-image training split;No binary values; No augmented.

For an original pixel $u \in \{0, \ldots, 255\}$, preprocessing is

$$x = 2\left(\frac{u}{255}\right) - 1,$$

giving a `float32` tensor in $[-1, 1]$. This range matches the generator's final Tanh activation. For display, pixels are mapped back using $(x + 1)/2$.

prepare_data.py acquires the raw files, adds the channel dimension, performs the transformation, and saves the training tensor. mnist_data.py loads it and returns **images only**. MNIST digit labels are used for data-distribution statistics, not as inputs to G or D, because the model is unconditional.

The loader shuffles each epoch and keeps the final incomplete batch: 469 batches with a final batch of 96 images for the full set, or 47 batches with a final batch of 112 for E0. Its random stream is separate from model initialization and training-noise sampling.

In [ ]:
```
show_figure(report, 'outputs/data/training_samples.png', width=560)
```


Figure 1. The first 64 real training images in their original order, displayed in [0,1].

3. Model modules and adversarial learning

3.1 Generator

The generator G receives $z \sim \mathcal{N}(0, I_{100})$. A fully connected layer creates a small spatial feature representation, and two transposed convolutions learn successive upsampling operations. The implementation is `Generator` in models.py. Here, B denotes the actual batch size.

Stage	Operation	Output shape	Function
Input	Gaussian noise	$B \times 100$	Supplies variation between generated samples
Projection	Linear(100, 6272), ReLU	$B \times 6272$	Learns an initial nonlinear feature representation
Reshape	Unflatten	$B \times 128 \times 7 \times 7$	Organizes the features spatially
Upsampling 1	ConvTranspose2d(128, 64), kernel 4, stride 2, padding 1	$B \times 64 \times 14 \times 14$	Learns spatial upsampling and feature mixing
Normalization	BatchNorm2d(64), ReLU	$B \times 64 \times 14 \times 14$	Normalizes intermediate channels and applies a nonlinearity

Stage	Operation	Output shape	Function
Upsampling 2	ConvTranspose2d(64, 1), kernel 4, stride 2, padding 1	$B \times 1 \times 28 \times 28$	Produces the grayscale image
Output	Tanh	$B \times 1 \times 28 \times 28$	Restricts generated pixels to $[-1, 1]$

The baseline generator has **765,761 trainable parameters**. E3 replaces the single BatchNorm layer with Identity, leaving the other layers unchanged. Its generator has **765,633 parameters**, a reduction of 128 channel-scale and bias parameters. BatchNorm running statistics are saved buffers, rather than trainable parameters.

3.2 Discriminator

The discriminator D extracts local image features, reduces spatial resolution, and outputs one **logit** per image. Both real and generated images use the same input shape and range.

Stage	Operation	Output shape	Function
Input	Grayscale image	$B \times 1 \times 28 \times 28$	Accepts a real image or $G(z)$
Feature block 1	Conv2d(1, 64), kernel 4, stride 2, padding 1; LeakyReLU(0.2)	$B \times 64 \times 14 \times 14$	Extracts local features and downsamples
Feature block 2	Conv2d(64, 128), kernel 4, stride 2, padding 1; LeakyReLU(0.2)	$B \times 128 \times 7 \times 7$	Builds a deeper image representation
Flatten	Flatten all non-batch dimensions	$B \times 6272$	Prepares features for the output layer
Classifier	Linear(6272, 1)	$B \times 1$	Outputs a real-versus-generated logit

1. LeakyReLU keeps a nonzero slope for negative inputs. D has **138,561 trainable parameters** and contains no BatchNorm or Dropout.
2. No final Sigmoid because `BCEWithLogitsLoss` incorporates that operation.

3.3 Losses and alternating updates

Let $\ell(s, t)$ denote binary cross-entropy applied to a logit s and target t through `BCEWithLogitsLoss`. The objectives are

$$L_D = \mathbb{E}_x \ell(D(x), 1) + \mathbb{E}_z \ell(D(G(z)), 0),$$

$$L_G = \mathbb{E}_{z'} \ell(D(G(z')), 1).$$

The two terms in D's loss are each averaged over their batch and then added, without dividing their sum by two. G uses the non-saturating objective, which encourages D to classify generated images as real.

train.py performs the following operations for every batch:

1. Generate fake images, evaluate real images against target 1 and detached fake images against target 0, and take one Adam step for D. Detaching prevents this loss from propagating into G.
2. Resample noise, freeze D's parameters, evaluate the new generated images against target 1, and take one Adam step for G. The computation through D remains differentiable with respect to its input.
3. Restore D's trainable state for the next batch.

This gives a D:G update ratio of **1:1**. There is no label smoothing, additional regularization, or mixed-precision training in these runs.

3.4 Sampling, logging, and checkpointing

Each epoch saves G/D losses, D's mean probabilities on real and generated images, the number of samples and updates, elapsed time, a fixed-noise sample grid, and a checkpoint. Epoch metrics are weighted by batch size, so the last 96-image batch does not receive the same total weight as a full 128-image batch. The logged D probabilities come from the **D-stage forward passes before its optimizer step**.

The 64 training-display noise vectors use a separate generator with seed **2026**. Sampling switches G to evaluation mode, disables gradient recording, and restores its previous mode afterward. This prevents display sampling from updating BatchNorm statistics or consuming the training-noise random stream.

Each checkpoint contains G/D weights, BatchNorm buffers, both Adam states, configuration, epoch, metrics, and random states. The main comparison consistently uses the **epoch-20 checkpoint**.

4. Experimental settings

The formal experiments use batch size **128**, latent dimension **100**, Adam betas **(0.5, 0.999)**, weight decay **0**, and training seed **42**. Each run creates fresh models, optimizers, and a shuffled loader. E2 changes only the common G/D learning rate; E3 changes only generator BatchNorm.

E0 uses a smaller subset and two epochs to exercise the complete training pipeline. It is excluded from the formal quality ranking. E1-E3 each complete 469 batches per epoch and **9,380 optimizer updates per network** over 20 epochs. The tables below are read from the recorded configuration and environment files.

In [ ]:
```
show_table(report, 'experiments')
show_table(report, 'environment')
```

5. Training process

5.1 Loss curves and discriminator responses

Figure 2 plots all 20 raw epoch means for each formal run. Initial fluctuations are followed by smaller changes in the epoch-level losses and discriminator probabilities. The baseline G loss remains near 1 during much of the later training, while its D loss is also near 1. E3 starts with a larger G loss and approaches a more stable range later.

In [ ]:
```
show_figure(report, 'outputs/comparison/seed_42/training_comparison.png')
show_table(report, 'losses')
```

Figure 2. Raw sample-weighted G/D losses and D-stage mean probabilities.

5.2 Generated images as training progresses

A 64-image grid was saved after every epoch. Figure 3 arranges the baseline checkpoints into the requested **5-row by 4-column** progression: rows correspond to epochs **1, 5, 10, 15, and 20**, and each column reuses the same noise vector. The four vectors are the first four from the fixed seed-2026 display set.

The first row contains loosely organized strokes. By epoch 5, the examples have clearer digit-like shapes. Later stages refine the strokes while largely retaining the identities associated with these inputs. This shows a learning trajectory;

In [ ]:
```
show_figure(report, 'outputs/comparison/seed_42/baseline_progress.png', width=54
```

Figure 3. Baseline generation progress. Rows are training stages and columns are fixed noise inputs.

6. Generated examples

Each epoch-20 generator produces 200 images from the same independent evaluation noise set, with seed **2027**. Figure 4 displays exactly the **first 64** images from each set.

A common noise input controls sampling across configurations, but it does not impose a digit identity. A particular position can therefore depict different digits

in E1 and E3 without that difference being an error.

In [ ]:
```
show_figure(report, 'outputs/comparison/seed_42/final_samples.png')
```

Figure 4. Epoch-20 examples for E1, E2, and E3, using the same evaluation inputs and fixed display range. No image was replaced or filtered based on quality.

Checkpoint-based generation demonstration

In [ ]:
```
#Reloads the saved E1 checkpoint and runs G on the first eight saved evaluation
show_checkpoint_demo(report, experiment='E1', count=8)
```

Figure 5. Eight examples regenerated by the report from the baseline checkpoint.

7. Human evaluation and sample diversity

The 200 images from each configuration were presented on annotation sheets with anonymous set identifiers A-C. I assigned a digit label 0-9 or `unreadable` by myself; Recognizability is the fraction assigned a digit label. Class coverage counts the distinct recognizable digit labels. Unreadable images remain in the 200-image denominator.

The baseline has 179 recognizable samples, compared with 168 for E2 and 164 for E3. Under this fixed 20-epoch budget, lowering the learning rate reduces the observed recognizability by **5.5 percentage points**, and removing generator BatchNorm reduces it by **7.5 percentage points**.

I did not label instances where the cause could not be identified, and since there are quite a few images that are difficult to distinguish, manual labeling may result in some errors.

In [ ]:
```
show_table(report, 'quality')
show_figure(report, 'outputs/comparison/seed_42/class_distribution.png')
```

Figure 6. Human label counts across all 200 images per run.

Digit 1 occurs only **4, 4, and 3** times in E1-E3, whereas digit 9 occurs **38, 37, and 46** times. The generated distributions are therefore not balanced.

Human-labeled failure examples

Figure 7 shows the first four `unreadable` examples per experiment in ascending sample-ID order. This deterministic selection documents human-identified failures without replacing them with more favorable outputs.

In [ ]:
```
show_failure_examples(report)
```

Figure 7. Human-labeled unreadable samples.

8. Training and inference time

Training time covers the epoch loop, including batch updates, per-epoch sample generation, checkpoint writes, metric writes, and progress output. It excludes data/model preparation, the epoch-0 preview, and final aggregate plotting. GPU work is synchronized at timing boundaries. The table reports the original measured times from the completed runs.

Inference time measures G generating a batch of **1,000 images** in evaluation mode under `no_grad`. Noise is allocated on the device before timing, using seed 2028.

In [ ]:
```
show_table(report, 'runtime')
```

Amortized time per image is the batch mean divided by 1,000.

9. Discussion, limitations, and conclusion

Experimental Approach: First, establish a moderately-sized, easily interpretable convolutional GAN baseline; then, conduct single-factor experiments to examine the effects of the learning rate and the generator's BatchNorm. Using an Unconditional GAN with a convolutional architecture

Data: The experiment used 60,000 training images from the original MNIST dataset, retaining the 1×28×28 image dimensions. No binarization or data augmentation was performed. The batch size was 128; the validation set consisted of 469 batches per round, with the final batch containing 96 images. The testing set consisted of 47 batches per round, with the final batch containing 112 images.

Training Setup: In the formal experiment, each group was trained for 20 rounds, with G and D updated 9,380 times each; the weights from the 20th round were ultimately used for all groups, and the top 64 images from the evaluation set are shown.

Network: The network architecture is described in Chapter 3 above. The sequence of operations for each batch is as follows:

1. Update D: Generate a fake image, apply `detach()` to the fake image, compute the loss between the true and generated images, and perform one Adam update for D.
2. Update G: Resample the noise, freeze D's parameters, compute the loss using the new fake image against target 1, and perform one Adam update for G.
3. Restore D's parameters, then next batch.

Some conclusions and summary:

- Figure 3: Shows the output of rounds 1, 5, 10, 15, and 20 for the same four noise patterns. As can be seen, the early results consist of blurry, loose strokes; by round 5, the digit shapes are already fairly clear; and subsequent rounds primarily focus on improving clarity.
- In the manual evaluation, the baseline E1 performed best, with recognition rates of 89.5%, 84%, and 82% for E1, E2, and E3, respectively.
- Small convolutional GANs are already capable of generating a variety of recognizable digits from noise.
- Reducing the joint learning rate and removing the generator's BatchNorm both decreased the recognizability rate within the fixed budget of this experiment.
- All three groups still exhibited blurriness, mixed character shapes, and skewed class distributions; neither generation quality nor diversity has been fully resolved.

These results support two qualified conclusions:

1. Learning rate experiments: After halving the shared learning rate, the number of distinguishable samples decreased by 11, a drop of 5.5 percentage points, and no improvement in quality was observed within the 20-round budget.
2. BatchNorm ablation: After removing BatchNorm from G, the number of distinguishable samples decreased by 15, a drop of 7.5 percentage points, which supports retaining BatchNorm in this baseline.
3. Class imbalance in manual annotation may be due to a skewed distribution of classes learned by the generator; at the same time, manual interpretation of some blurry images may also affect the statistics. In the actual training set, there aren't particularly many instances of class 9. The statistics for the original training set show class balance and are unrelated to this issue.

Possible reasons for suboptimal results: using only a single training seed, evaluating only 200 samples per configuration, relying on only one human annotator, a lack of quantitative measures of intra-class replication, and unstable runtime conditions.

10. Code organization and reproducibility

File	Responsibility and connection
prepare_data.py	Acquires raw MNIST, normalizes images, saves the complete tensor and fixed debug indices, and exports data statistics
mnist_data.py	Loads processed tensors and supplies image-only, shuffled batches to training
models.py	Implements the G/D modules described in Section 3
train.py	Defines E0-E3 configurations, runs alternating Adam updates, and writes metrics, fixed samples, and checkpoints

File	Responsibility and connection
evaluate.py	Reloads epoch-20 weights, samples common evaluation noise, times inference, and aggregates human labels
report_utils.py	Loads recorded results and renders the short tables, image displays, and checkpoint demonstration used here
debug.ipynb	Maintains the project's data, model, training, and evaluation debugging workflow

The report's tables read comparison/seed_42/summary.json, with the human labels in annotations.csv. Each formal run retains its configuration, raw epoch metrics, 20 checkpoints, fixed-noise samples, and 30 raw inference measurements. comparison.csv and class_counts.csv provide machine-readable summaries.

To open this report, use the project directory and select any compatible Python kernel. Install the packages listed in requirements.txt in that environment. Reading the saved outputs requires only this executed notebook. Rerunning the short cells requires the accompanying Python modules and saved `outputs/` artifacts, including the E1 epoch-20 checkpoint for the demonstration. The recorded training/benchmark environment describes the original runs; it is not a requirement for the current kernel.

Training reproduction is separate from report execution. For example, the following command starts a new E1 run in a distinct directory; replace the experiment and output directory for E2 or E3:

```
python train.py --experiment E1 --device auto --output-dir
outputs/E1/seed_42_repeat
```

The report does not execute that command. Its generation demonstration is a forward pass from existing weights, and its runtime table preserves the original benchmark measurements.